

halfedge：内置属性模块设计文档

src/builtin_attrs.rs

halfedge 库

2026-07-05

目录

1 模块定位	1
2 为什么需要 Newtype 包装?	2
3 API 表	2
4 属性数据流	3
5 OBJ 属性感知 IO	3
5.1 解析: parse_obj_with_attrs	3
5.2 序列化: format_obj_with_attrs	4
6 批量安装: install_vertex_attrs	4
7 与 property.rs 的关系	5
8 退化守卫	6
9 使用示例	6
9.1 为 icosphere 计算顶点法向并写出 OBJ	6
9.2 从带属性的 OBJ 加载	6
9.3 批量安装自定义属性	6
10 测试覆盖	7

1 模块定位

`builtin_attrs.rs` 在 `property.rs` 泛型属性系统之上，为常见几何属性（顶点法向 / UV / 颜色 / 面法向）提供强类型 Newtype 包装与便捷函数，并实现 OBJ 属性感知 IO（v/vt/vn 解析与序列化）。

2 为什么需要 Newtype 包装?

property.rs 的属性系统基于 `TypeId` 区分属性类型:

属性键 = `TypeId::of::<T>()`

`TypeId` 由 Rust 编译器为每个 `'static` 类型自动分配, 相同类型共享同一 ID。问题: 若想为顶点同时附加 `[f64; 3]` 类型的「法向」与「颜色」, 二者 `TypeId` 相同, 会互相覆盖。

解决方案: 用 Newtype 包装, 让每个语义属性有独立类型:

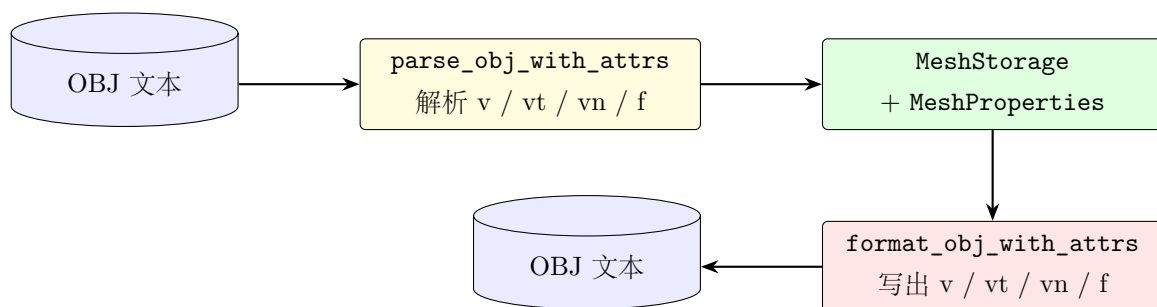
```
1 #[derive(Debug, Clone, Copy, Default, PartialEq)]
2 pub struct VertexNormal(pub [f64; 3]);
3
4 #[derive(Debug, Clone, Copy, Default, PartialEq)]
5 pub struct VertexColor(pub [f64; 3]);
6
7 #[derive(Debug, Clone, Copy, Default, PartialEq)]
8 pub struct VertexUv(pub [f64; 2]);
9
10 #[derive(Debug, Clone, Copy, Default, PartialEq)]
11 pub struct FaceNormal(pub [f64; 3]);
```

`VertexNormal` 与 `VertexColor` 现在是不同类型, `TypeId::of::<VertexNormal>() ≠ TypeId::of::<VertexColor>()` 可以共存于同一 `MeshProperties` 容器。

3 API 表

类别	API	语义
Newtype	<code>VertexNormal([f64;3])</code>	顶点法向（单位向量）
	<code>VertexColor([f64;3])</code>	顶点颜色 RGB，每通道 $\in [0,1]$
	<code>VertexUv([f64;2])</code>	顶点 UV 纹理坐标
	<code>FaceNormal([f64;3])</code>	面法向（单位向量）
注册	<code>add_vertex_normal(&mut props)</code>	注册顶点法向属性，返回 <code>Vec<[f64;3]></code>
	<code>add_vertex_colors(&mut props)</code>	注册顶点颜色属性
	<code>add_vertex_uvs(&mut props)</code>	注册顶点 UV 属性
	<code>add_face_normal(&mut props)</code>	注册面法向属性
批量计算	<code>populate_vertex_normal(mesh, props, h)</code>	用 <code>vertex_normal</code> 填充所有面
	<code>populate_face_normal(mesh, props, h)</code>	用 <code>face_normal</code> 填充所有面
收集	<code>collect_vertex_normal(mesh, props, h) → Vec<[f64;3]></code>	按顶点顺序提取法向
	<code>collect_vertex_uvs(mesh, props, h) → Vec<[f64;2]></code>	按顶点顺序提取 UV
	<code>collect_vertex_colors(mesh, props, h) → Vec<[f64;3]></code>	按顶点顺序提取颜色
批量安装	<code>install_vertex_attrs(props, normals, uvs, colors, ids)</code>	一次性注册并填充
OBJ 属性感知	<code>parse_obj_with_attrs(text) → (Mesh, Props)</code>	解析 OBJ 并提取 vt/vn
	<code>format_obj_with_attrs(mesh, props, n_h, uv_h) → String</code>	序列化带 vt/vn 的 OBJ

4 属性数据流



5 OBJ 属性感知 IO

5.1 解析：parse_obj_with_attrs

输入：OBJ 文本，含 `v / vt / vn / f` 行。输出：`(MeshStorage, MeshProperties)`，属性系统已注册 `VertexNormal` 与 `VertexUv`（仅在输入含对应行时）。

面行格式 支持 4 种：

- `f v1 v2 v3` ——仅顶点；

- `f v1/vt1 v2/vt2 v3/vt3` ——顶点 + UV;
- `f v1/vt1/vn1 v2/vt2/vn2 v3/vt3/vn3` ——顶点 + UV + 法向;
- `f v1//vn1 v2//vn2 v3//vn3` ——顶点 + 法向 (无 UV)。

属性映射 每个面顶点 (`vt`, `vn`) 索引 (1-based) 映射到顶点:

1. 收集所有 `vt` 行到 `texcoords: Vec<[f64;2]>`;
2. 收集所有 `vn` 行到 `normals: Vec<[f64;3]>`;
3. 遍历每个面顶点 (`v_idx`, `vt_idx`, `vn_idx`): `vert_normal[v_idx] = normals[vn_idx - 1]`;
4. 按顶点顺序注册属性并填充 `MeshProperties`。

同顶点多面引用 同一顶点可能被多个面引用, 后面的面覆盖前面的 (OBJ 标准允许同顶点在不同面有不同 UV/法向, 本实现按顶点而非按面顶点存储属性, 故取最后一次赋值)。

5.2 序列化: `format_obj_with_attrs`

输出顺序:

1. 所有 `v` 行 (顶点位置);
2. 所有 `vt` 行 (若 `uv_handle` 为 `Some`);
3. 所有 `vn` 行 (若 `normal_handle` 为 `Some`);
4. 所有 `f` 行, 按 `v/vt/vn` 格式输出 (依据已注册属性选择)。

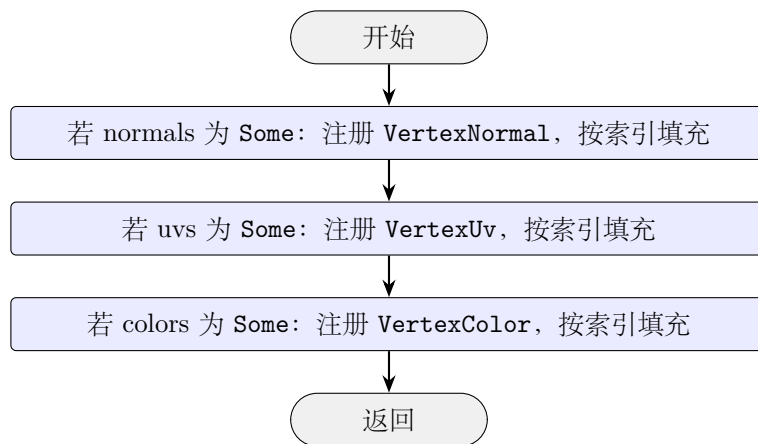
面行格式选择

- 同时有 UV 和法向: `f v/vt/vn v/vt/vn v/vt/vn`;
- 仅有 UV: `f v/vt v/vt v/vt`;
- 仅有法向: `f v//vn v//vn v//vn`;
- 都无: `f v v v`。

未设置属性的顶点写 0 占位 (`vt 0 0` 或 `vn 0 0 0`)。

6 批量安装: `install_vertex_attrs`

`install_vertex_attrs(props, normals, uvs, colors, vertex_ids)` 用于从外部数据 (如其他 IO 库解析结果) 一次性注册并填充属性:



部分输入：任一参数为 `None` 则跳过对应属性，`has_vertex_prop::()` 对应返回 `false`。

7 与 `property.rs` 的关系

维度	<code>property.rs</code> (泛型)	<code>builtin_attrs.rs</code> (内置)
类型	任意 <code>T: 'static</code>	固定 4 种 Newtype
句柄	<code>PropertyHandle<T></code>	同左 (直接复用)
存储	<code>HashMap<TypeId, Box<dyn ErasedProperty>></code>	同左
注册	<code>add_vertex_prop::<t>()</t></code>	<code>add_vertex_normal(&mut props)</code>
读写	<code>get/set_vertex_prop</code>	同左
语义	无 (用户自定义)	法向 / UV / 颜色 / 面法向
IO 联动	无	<code>parse_obj_with_attrs</code> 等

`builtin_attrs` 完全建立在 `property.rs` 之上，不修改其底层存储。用户可同时使用两者：自定义泛型属性与内置属性互不冲突。

8 退化守卫

函数	退化场景	处理
<code>populate_vertex_normal</code>	顶点法向不可计算（孤立 / 边界退化）	跳过该顶点
<code>populate_face_normal</code>	面法向不可计算（共线）	跳过该面
<code>populate_face_normal</code>	空网格	安全返回（无 panic）
<code>collect_vertex_normal</code>	顶点未设置法向	填 [0, 0, 0]
<code>collect_vertex_colors</code>	顶点未设置颜色	填 [1, 1, 1]（白）
<code>collect_vertex_uv</code>	顶点未设置 UV	填 [0, 0]
<code>parse_obj_with_attrs</code>	OBJ 无 <code>vn</code> 行	不注册 <code>VertexNormal</code>
<code>parse_obj_with_attrs</code>	面索引越界	返回 <code>IndexOutOfRange</code> 错误
<code>parse_obj_with_attrs</code>	面顶点数 < 3	返回 <code>NotTriangular</code> 错误

9 使用示例

9.1 为 icosphere 计算顶点法向并写出 OBJ

```
1 use halfedge::*;
2 use halfedge::builtin_attrs::*;
3
4 let mesh = test_util::build_icosphere(2);
5 let mut props = MeshProperties::new();
6 let n_handle = add_vertex_normals(&mut props);
7 populate_vertex_normals(&mesh, &mut props, n_handle);
8
9 let obj = format_obj_with_attrs(&mesh, &props,
10     Some(n_handle), None);
11 // obj      v / vn / f      f      "f v//vn v//vn v//vn"
```

9.2 从带属性的 OBJ 加载

```
1 let text = std::fs::read_to_string("model.obj")?;
2 let (mesh, props) = parse_obj_with_attrs(&text)?;
3 // props      VertexNormal      VertexUv      OBJ      vn / vt
```

9.3 批量安装自定义属性

```
1 let mesh = build_icosphere(0);
2 let ids: Vec<VertexId> = mesh.vertex_ids().collect();
3 let mut props = MeshProperties::new();
4
5 let normals: Vec<[f64; 3]> = ids.iter().map(|_| [0.0, 1.0, 0.0]).collect();
6 let uvs: Vec<[f64; 2]> = ids.iter().enumerate()
```

```

7      .map(|(i, _)| [i as f64 * 0.1, 0.0])
8      .collect();
9
10 install_vertex_attrs(&mut props, Some(&normals), Some(&uvs), None, &ids);
11 assert!(props.has_vertex_prop::<VertexNormal>());
12 assert!(props.has_vertex_prop::<VertexUv>());
13 assert!(!props.has_vertex_prop::<VertexColor>());

```

10 测试覆盖

测试名	验证点
vertex_normal_roundtrip	icosphere 顶点法向为单位向量
face_normal_roundtrip	所有面法向已写入
vertex_color_default_is_white	未设置颜色返回默认白
vertex_uv_set_and_get	UV 设置后可读取
install_vertex_attrs_handles_partial_input	仅传法向不注册 UV/Color
install_vertex_attrs_all_three	三种属性同时安装
newtype_default_is_zero	Newtype 默认值为零
missing_handle_returns_none	未注册句柄读取返回 None
populate_face_normal_skips_invalid	空网格不 panic

src/builtin_attrs.rs 共 9 个单元测试，全库共 371 个。